

Фронтенд: что откуда появляется

Информационная система городской телефонной сети

Как из горстки конфигов и универсальных компонентов рождается весь интерфейс

Содержание

1	Картина целиком	2
1.1	Поток данных (как страница получает данные)	2
2	Точка входа: <code>main.ts</code> и <code>App.vue</code>	2
3	Маршрутизация: <code>router/index.ts</code>	3
4	Состояние: <code>Pinia</code> -сторы	3
5	API-слой: <code>api/</code>	3
6	Конфиг-движок — сердце фронта	4
6.1	<code>config/enums.ts</code> — справочники значений	4
6.2	<code>config/resources.ts</code> — описания 16 справочников	4
6.3	<code>config/analytics.ts</code> — 13 запросов варианта	4
6.4	Как из конфигов рождается интерфейс	4
7	Универсальные экраны и каталог страниц	4
8	Оболочки (<code>layouts</code>) — где появляется «что видно»	5
9	Сборка и отдача: <code>Vite</code> и <code>nginx</code>	5
10	Сводка: что откуда появляется	6
11	Шпаргалка к защите (по фронту)	6

1 Картина целиком

Фронтенд — это **SPA** (single-page application): один HTML-файл и JavaScript, который сам рисует все страницы и общается с бэкендом по HTTP (JSON). Перегрузки страниц нет — меняется только содержимое.

Технология	Зачем
Vue 3 + TypeScript	компоненты интерфейса (.vue), типобезопасность
Vite	дев-сервер с горячей перезагрузкой и сборка в статику
Vue Router	маршруты (какой компонент на каком URL) + охрана доступа
Pinia	хранилище состояния (кто вошёл, его права)
Element Plus	готовые UI-компоненты: таблицы, формы, диалоги, меню
axios	HTTP-клиент к API

Главная идея проекта: операторская часть **не написана страница за страницей**. Есть **конфиги-описания** (какие сущности, какие поля, какие запросы) и **универсальные компоненты**, которые по этим описаниям сами рисуют таблицы, формы и меню. Добавить сущность — значит дописать строчку в конфиг, а не верстать новый экран.

1.1 Поток данных (как страница получает данные)

Компонент (views/*.vue)
| вызывает функцию
▼
api/*.ts → api/client.ts (axios: baseUrl '/api', шлёт cookie)
|
dev: Vite-прокси /api → http://localhost:8080 prod: nginx /api → backend:8080
▼
Ахуй (бэкенд) —JSON—> обратно в компонент (ref / Pinia)

Где смотреть. Корень фронта — frontend/. Исходники — frontend/src/: main.ts (вход), App.vue, router/, stores/, api/, config/, layouts/, views/.

2 Точка входа: main.ts И App.vue

main.ts создаёт приложение и **подключает плагины** — после этого они доступны во всех компонентах:

```
const app = createApp(App)
for (const [name, icon] of Object.entries(ElementPlusIconsVue)) app.component(name,
icon) // иконки
app.use(createPinia()) // хранилище
app.use(router) // маршруты
app.use(ElementPlus) // UI-кит + стили
app.use(VueQueryPlugin) // (подключён на будущее; экраны грузят данные напрямую через
api)
app.mount('#app')
```

App.vue предельно прост — это «рамка», в которую роутер подставляет текущую страницу:

```
<template><router-view /></template>
```

Важно. В main.ts также стоит «глушилка» безобидной ошибки ResizeObserver loop ..., которую иногда эмитит браузер при перерисовке таблиц Element Plus (см. гайд/историю). Она ничего не ломает.

3 Маршрутизация: `router/index.ts`

Роутер сопоставляет URL и компонент. Страницы грузятся **лениво** (`() => import(...)`) — код экрана подтягивается, только когда на него заходят. Маршруты разбиты на **три зоны**:

Зона	Что внутри
Публичная (<code>meta.public</code>)	лендинг /, вход сотрудника <code>/staff/login</code> , вход абонента <code>/portal/login</code>
Операторская (<code>/app, MainLayout</code>)	<code>dashboard</code> , <code>crud/:resource</code> , <code>applications</code> , <code>analytics</code> , <code>raw-query</code> , <code>settings</code> , <code>admin/*</code>
Кабинет (<code>/portal, PortalLayout</code>)	<code>portal-home</code> , <code>apply</code> , <code>line/:id</code>

Перед каждым переходом срабатывает **охранник** `router.beforeEach`: публичные пускает; для `/app` требует входа сотрудника и (если у страницы есть `meta.perm`) нужного права; для `/portal` — входа абонента. Подробности — в `docs/auth.pdf`.

Откуда берётся. Один параметрический маршрут `crud/:resource` обслуживает **все** справочники: `:resource` (`cities`, `pbx`, ...) — это `key` из конфига ресурсов. Поэтому новый справочник не требует нового маршрута.

4 Состояние: `Pinia`-сторы

Два независимых хранилища держат «кто вошёл»:

Стор	Что хранит и умеет
<code>stores/auth.ts</code> (сотрудник)	<code>user</code> (+ права); геттер <code>can(perm)</code> ; <code>fetchMe / login / logout</code>
<code>stores/customer.ts</code> (абонент)	<code>customer</code> ; геттер <code>fullName</code> ; <code>fetchMe / login / register / logout</code>

`can(perm)` — главный инструмент UI: `is_superuser || permissions.includes(perm)`. По нему скрываются пункты меню и кнопки. `fetchMe` нужен, чтобы после перезагрузки страницы понять, жива ли сессия (`cookie` уже в браузере).

5 API-слой: `api/`

Все обращения к бэкенду — тонкие функции поверх одного `axios`-клиента.

```
// api/client.ts
const api = axios.create({ baseURL: '/api', withCredentials: true })
// + перехватчик: любую ошибку превращает в человеко-читаемый русский текст (humanMessage)
```

- `baseURL: '/api'` — все пути относительные; куда реально идёт `/api`, решают прокси (см. раздел 9).
- `withCredentials: true` — слать `cookie` сессии (без этого каждый запрос был бы «не авторизован»).
- Перехватчик ошибок даёт единый русский текст (`401` → «Требуется авторизация», нет связи и т.д.).

Дальше — модули по областям (каждый просто дёргает `api.get/post/...`):

Модуль	Эндпоинты
<code>api/crud.ts</code>	generic CRUD <code>/path</code> , аналитика, <code>raw-query</code> , настройки биллинга
<code>api/ops.ts</code>	очередь заявок и подключение (<code>/ops/...</code>)
<code>api/portal.ts</code>	личный кабинет (<code>/portal/...</code>)
<code>api/admin.ts</code>	пользователи, роли, права (<code>/admin/...</code>)
<code>api/types.ts</code>	общие типы (<code>Row</code> , <code>Page<T></code> , <code>CurrentUser</code>)

6 Конфиг-движок — сердце фронта

Здесь и кроется ответ «откуда что появляется». Три файла-описания превращаются в готовый UI универсальными компонентами.

6.1 config/enums.ts — справочники значений

Списки вариантов (Opt = { label, value }) для выпадающих списков: типы линий, статусы, пол, категории и т.д. **Подписи русские**, значения совпадают с enum в БД.

```
export const lineTypeOptions: Opt[] = [
  { label: 'Основной', value: 'main' }, { label: 'Параллельный', value: 'parallel' }, ...
]
```

6.2 config/resources.ts — описания 16 справочников

Каждый ресурс — это key, path (REST-путь), title, perm (право), columns (что в таблице) и fields (что в форме). Поле формы бывает text/number/select/switch/date/reference.

```
{ key: 'subscribers', path: 'subscribers', title: 'Абоненты', perm: 'subscriber',
  columns: [ { prop: 'last_name', label: 'Фамилия' }, ... ],
  fields: [ t('last_name', 'Фамилия', true),
            sel('category', 'Категория', E.categoryOptions),
            ref('phone_number_id', 'Номер', 'phone-numbers', numberLabel, true) ] }
```

- **select** — выбор из enums.ts; **reference** — выбор из **другого справочника** (по refPath), с человекочитаемой подписью (refLabel). Так в формах не вводят «голый id», а выбирают из списка.

6.3 config/analytics.ts — 13 запросов варианта

Каждый запрос — key, title, path и список params (фильтры с типами). Из этого строится форма фильтров на странице аналитики.

6.4 Как из конфигов рождается интерфейс

Откуда берётся.

- **Пункты меню «Справочники»** ← resources (только те, где есть право perm:read).
- **Таблица справочника** ← columns; **форма** ← fields; всё рисует один CrudView.vue.
- **Выпадающие списки в форме** ← select (из enums.ts) или reference (подгружает другой справочник).
- **Читаемые подписи в таблице** (id → имя, active → «Активен», true → «Да») ← CrudView сопоставляет колонку с полем.
- **Страница «Аналитика»** и её фильтры ← analyticsQueries; рисует AnalyticsView.vue.

Важно. Рецепт «добавить справочник на фронте»: дописать один объект в resources.ts. Таблица, форма, выпадающие списки и пункт меню появятся **сами** — отдельный экран писать не нужно.

7 Универсальные экраны и каталог страниц

CrudView.vue (таблица + пагинация + диалог-форма) и AnalyticsView.vue (выбор запроса + фильтры + результат) — это «движки», работающие по конфигу. Остальные экраны — точечные.

Экран	Что делает / откуда данные
LandingView	выбор «кабинет / сотрудник» (публичная)
LoginView, portal/PortalLoginView	формы входа; пишут в Pinia-стор
DashboardView	приветствие + профиль из auth.user
CrudView	любой справочник по resources.ts (16 ресурсов)

Экран	Что делает / откуда данные
AnalyticsView	13 запросов по analytics.ts; колонки результата — по ключам ответа
RawQueryView	одно поле SQL → /api/raw-query (READ ONLY, 5 с)
SettingsView	настройки биллинга (одна запись)
ApplicationsView	очередь заявок + кнопка «Подключить» (/ops)
admin/UsersView, admin/RolesView	пользователи и роли/права (/admin)
portal/PortalDashboard, PortalApply, PortalLine	кабинет: линии, заявка, звонки/межгород/оплата (/portal)

Важно. Таблицы аналитики и SQL-консоли **не знают заранее** своих колонок: компонент берёт Object.keys первой строки ответа и строит столбцы динамически. Поэтому любой новый запрос отрисуется без правок.

8 Оболочки (layouts) — где появляется «что видно»

Layout	Роль
MainLayout (/app)	боковое меню + шапка оператора; пункты скрыты через v-if="auth.can(...)"
PortalLayout (/portal)	простая шапка кабинета (имя абонента, «Выйти»)

```
// MainLayout: меню справочников строится из конфига и прав
const visibleResources = computed(() => resources.filter(r) =>
auth.can(`${r.perm}:read`)))
...
<el-menu-item v-if="auth.can('analytics:read')" index="/app/analytics">Аналитика</el-menu-item>
```

Важно. v-if="auth.can(...)" — это **удобство**, а не защита: пункт просто не показывают. Настоящая проверка прав — на сервере (см. docs/auth.pdf).

9 Сборка и отдача: Vite и nginx

В деве (npm run dev): Vite поднимает SPA на :5173 и **проксирует** /api на бэкенд :8080. Благодаря прокси фронт и API — **один origin**, cookie работают без возни с CORS.

```
// vite.config.ts
server: { port: 5173, proxy: { '/api': { target: 'http://localhost:8080', changeOrigin: true } } }
resolve: { alias: { '@': './src' } } // '@/...' = 'src/...'
```

В проде (Docker): npm run build (vue-tsc проверяет типы + vite build) собирает статику, её отдаёт **nginx**. Он же проксирует /api, /swagger-ui, /api-docs на контейнер бэкенда, а на всё остальное отдаёт index.html:

```
location /api/ { proxy_pass http://backend:8080; }
location / { try_files $uri $uri/ /index.html; } # SPA-роутинг
```

Важно. `try_files ... /index.html` — ключ для SPA: на любой «глубокий» URL (`/app/crud/pbx`) сервер отдаёт тот же `index.html`, а дальше маршрут разбирает Vue Router в браузере. Иначе при перезагрузке такой страницы был бы 404.

10 Сводка: что откуда появляется

Элемент на экране	Откуда взялся
Пункт меню справочника	строка в <code>config/resources.ts</code> (+ право <code>perm:read</code>)
Колонки и форма справочника	<code>columns / fields</code> того же ресурса → <code>CrudView.vue</code>
Выпадающий список в форме	<code>select</code> (из <code>enums.ts</code>) или <code>reference</code> (другой справочник)
Имена вместо <code>id</code> / «Да-Нет» / <code>enum</code> по-русски	<code>cellLabel</code> в <code>CrudView</code> (по описанию поля)
Страница и фильтры аналитики	<code>config/analytics.ts</code> → <code>AnalyticsView.vue</code>
Столбцы результата аналитики/SQL	ключи первой строки JSON-ответа (динамически)
Видимость пунктов меню/кнопок	<code>auth.can(perm)</code> из <code>Pinia</code> (права из БД)
Имя пользователя, бейдж «Суперадмин»	<code>auth.user</code> (ответ <code>/api/auth/me</code>)
Куда идёт <code>/api</code>	Vite-прокси (дев) или <code>nginx</code> (прод) → бэкенд <code>:8080</code>
Русский текст ошибки	перехватчик <code>humanMessage</code> в <code>api/client.ts</code>

11 Шпаргалка к защите (по фронту)

Вопрос	Короткий ответ + где код
Как устроен фронт?	Vue 3 SPA; маршруты (<code>router</code>), состояние (<code>pinia</code>), API (<code>axios</code>), UI (<code>Element Plus</code>).
Почему мало кода на 16 справочников?	Конфиг-движок: <code>config/resources.ts</code> + один <code>CrudView.vue</code> рисуют таблицу/форму/меню.
Как добавить справочник?	Дописать объект в <code>resources.ts</code> — экран и пункт меню появятся сами.
Откуда выпадающие списки?	<code>select</code> → <code>enums.ts</code> ; <code>reference</code> → другой справочник (выбор вместо ввода <code>id</code>).
Как защищён UI?	<code>auth.can(perm)</code> прячет пункты/кнопки; реально решает сервер.
Как фронт общается с API?	<code>api/client.ts</code> (<code>axios</code> , <code>/api</code> , <code>cookie</code>); прокси <code>Vite/nginx</code> ведёт на <code>:8080</code> .
Почему <code>cookie</code> работают без CORS?	Прокси делает фронт и API одним <code>origin</code> (и в деве, и в проде).
Почему «глубокие» URL не 404?	<code>nginx try_files</code> → <code>index.html</code> ; маршрут разбирает Vue Router.

См. также: авторизация — `docs/auth.pdf`, CRUD изнутри — `docs/crud.pdf`, операции — `docs/operations.pdf`.